

Book Nook Implementation Overview
Group 12 COSC 360 Final Project
Gurkirn Kaur, Onyx Cheyne, Sawyer Cairns, Zheng Xue

Introduction

This project is a full-stack web application designed to facilitate book sharing and discussion through a virtual platform. The system integrates client-side and server-side technologies, real-time updates, and database persistence.

Features Implemented

1. Core Features

- User registration with image upload.
- Secure login system.
- Profile viewing and editing.
- Book posting system.
- Real-time commenting system.
- Real-time user chatting and reservations
- Admin dashboard with elevated permissions.

2. Admin Features

- Search users by name, ID, or email.
- Search listings by name or ID.
- Delete users or listings.

3. Database Features

- User data stored securely.
- Book listings stored with metadata.
- Image links stored in database.

System Architecture

1. Frontend (Client-Side)

- Built with HTML, CSS, and JavaScript
- Handles: UI rendering, form validation, and asynchronous API calls.

2. Backend (Server-Side)

- Build using Node.js/Express
- Handles: authentication, API endpoints, business logic, and database communication

3. Database

- Uses MongoDB for storage
- Collections include: users, books, and comments

File Structure

1. Frontend

/public - Includes: HTML pages (index, login, profile), CSS styling pages, and JavaScript scripts for validation and API calls.

2. Backend

/server - Includes: app.js as main server entry point, routes/ for API endpoints, controllers/ for logic handling, and models/ for database schemas.

3. Database Models

User.js - stores user credentials and images

Book.js - stores item details

Comment.js - replies to listings

4. Testing

/tests - This folder includes unit testing, API tests, and edge case testing.

Security Implementation

1. Server-Side Security

Passwords hashed using bcrypt.

Environment variables used for sensitive data.

Input sanitization prevents injection attacks.

File uploads restricted to valid image formats.

Middleware to securely acquire user Ids, and to check if a user is an admin.

2. Client-Side Security

Input validation for all forms.

Password fields masked.

Restricted input formats.

Error Handling

The system includes validation for all user inputs. The API responses include proper status codes, and the frontend displays meaningful error messages.

Testing and Deployment

1. Testing

Covers: core functionality, edge cases, and critical user flows.

Test cases are stored in the /tests directory.

Proof of testing can be found in the 'Documentation' folder

2. Version Control

There is a GitHub repository with consistent commits.

3. Docker Deployment

This is a fully containerized application that runs with a single command.

The services include a web server and database.

Limitations

1. The known limitations are:

- Limited scalability for very large user bases.

- Basic UI/UX.

- Image storage uses base 64 encoded images in the database rather than cloud bucket / blob storage.

- No advanced recommendation system.